WARWICK

Advanced Computer Graphics

Ray Tracing

Dr. Tom Bashford-Rogers

Overview

- Rays and Primitives
- Generating Rays
- First speedups

Advanced Computer Graphics!

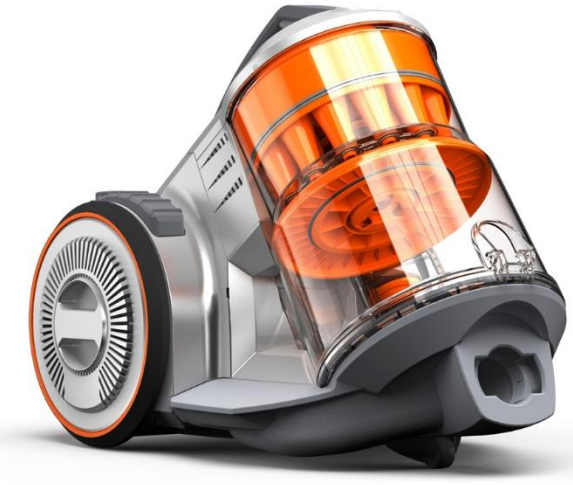
- Want to create realistic images
- Need to understand
  - How to simulate light
  - How to represent materials and light sources
  - How to do this quickly

Raytracing

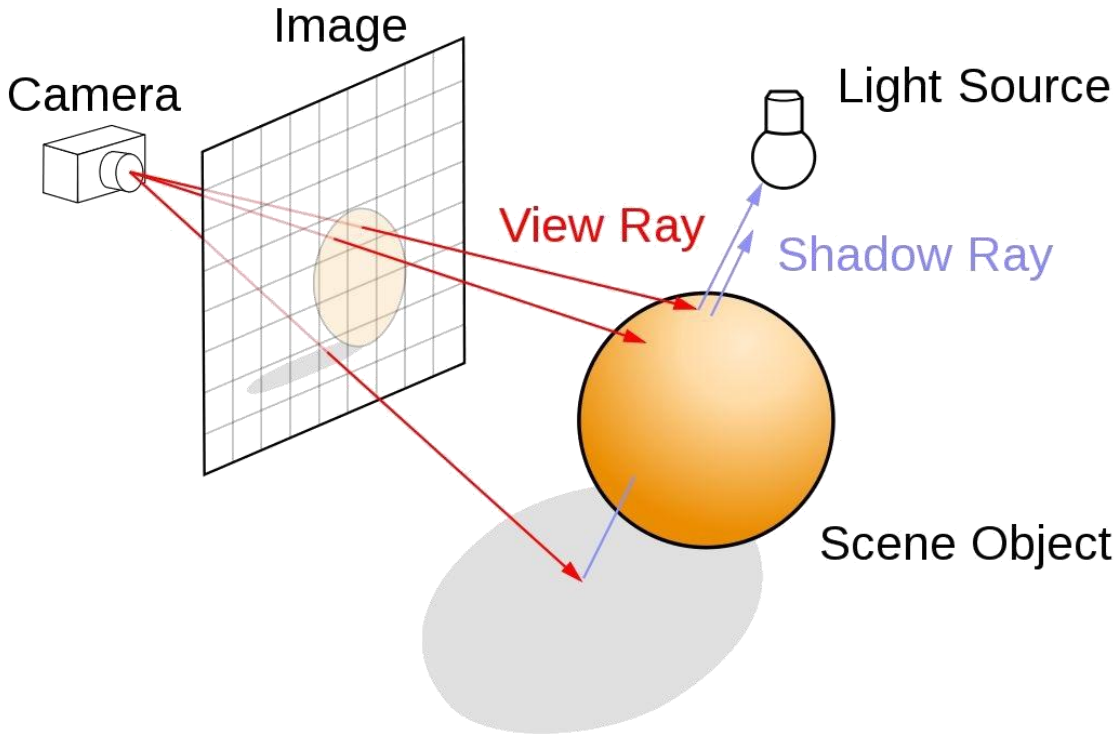
- Want to simulate the physical behavior of light
- Traces the path of rays from the eye (camera) to the scene
- Creates highly realistic images:
  - Shadows, reflections, and refractions

Raytracing

- Applications



Raytracing

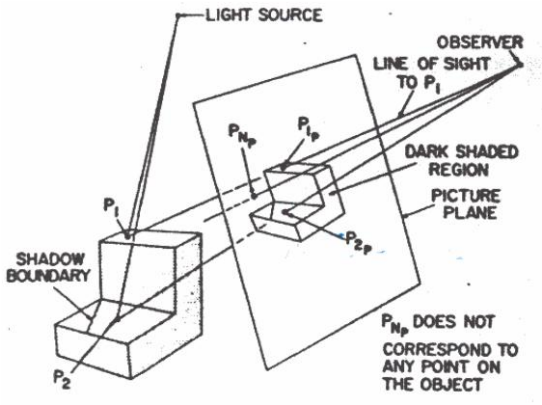


Raytracing

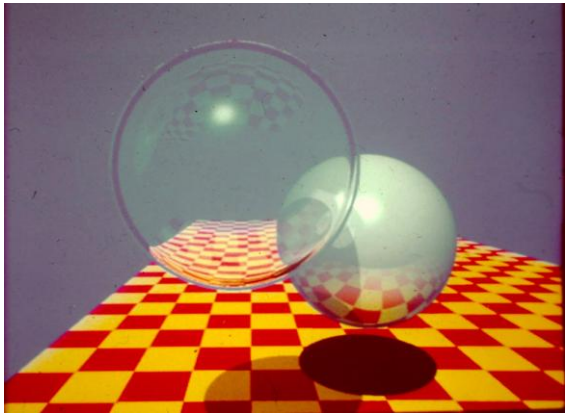
- Iterate over all pixels
- Pick a point in the pixel
- Generate rays
- Intersect scene geometry
- Shade

Raytracing

- History



Appel, A. (1968). “Some Techniques for Shading and Ray Tracing.”



Whitted, T. (1980). “An Improved Illumination Model for Shaded Display.”

Cook, R. L., Porter, T., & Carpenter, L. (1984). “Distributed Ray Tracing.”

Codebase

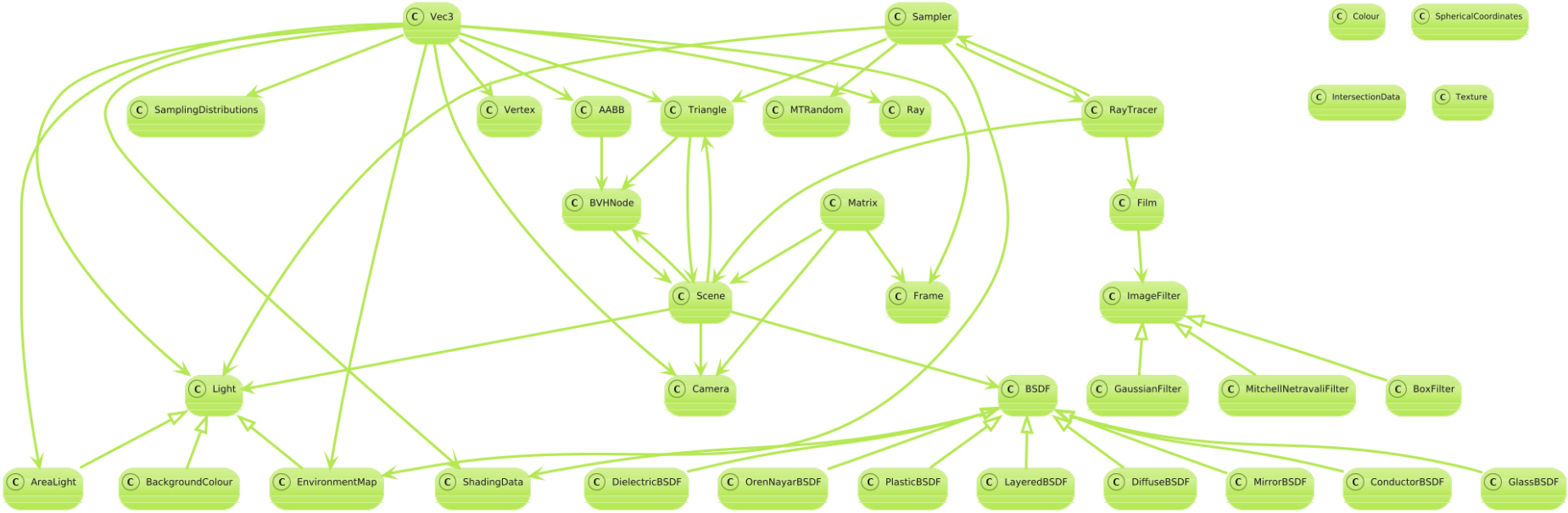
- Will create physically based rendering system
- Will build on a codebase:
  - Link:

    https://github.com/MSCGamesTom/RTBase.git

Codebase

- Comes with several scenes
  - Moodle
  - Examples (and more)

Codebase


Vec3
Sampler
Colour
SphericalCoordinates
SamplingDistributions
Vertex
AABB
Triangle
MTRandom
Ray
RayTracer
IntersectionData
Texture
BVHNode
Matrix
Film
Scene
Frame
ImageFilter
Light
Camera
BSDF
GaussianFilter
MitchellNetravaliFilter
BoxFilter
AreaLight
BackgroundColour
EnvironmentMap
ShadingData
DielectricBSDF
OrenNayarBSDF
PlasticBSDF
LayeredBSDF
DiffuseBSDF
MirrorBSDF
ConductorBSDF
GlassBSDF

Codebase

- Main:
  - Handles input
  - Command line options
    - Different scenes: -scene SCENE_NAME
    - Save to file: -outputFilename FILENAME.hdr
    - Render up to SPP: -SPP NUMBER

Codebase

- Includes a camera
  - WASD (Forward, Backward, Left, Right)
  - QE (Down, Up)
  - Can modify scene / filename / SPP in code

```
std::string sceneName = "cornell-box";
std::string filename = "Render.pfm";
unsigned int SPP = -1;
```

Codebase

- Saves rendered images to file
  - P saves HDR image (more on this later)
  - L saves LDR image displayed on screen
- Prints seconds per frame(!)

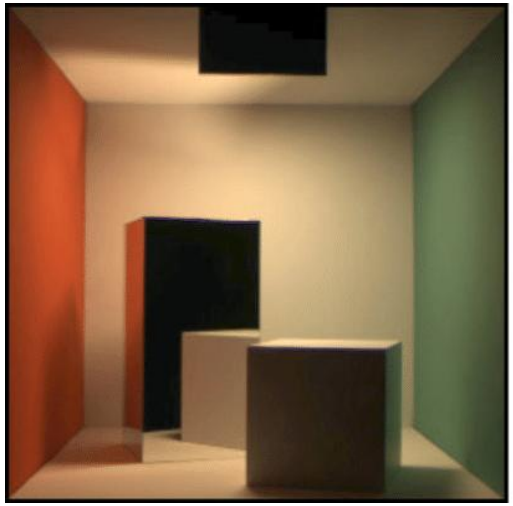
Codebase

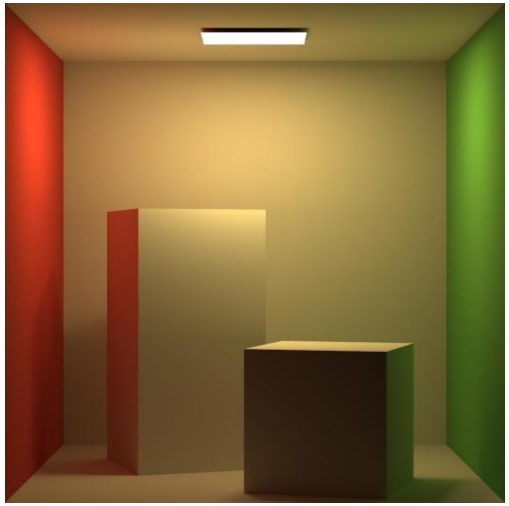
- Header files provide functionality
  - You will need to add to it

```
Renderer/
|— Core.h
|— Renderer.h
|— Sampling.h
|— Scene.h
|— SceneLoader.h
|— Lights.h
|— Geometry.h
|— Imaging.h
|— Materials.h
|— GEMLoader.h
|— GamesEngineeringBase.h
```

Scenes

- Will need Cornell Box

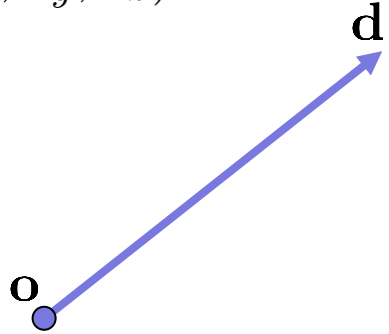




Rays

- Specified by an origin $\mathbf{o} = (o_x, o_y, o_z)$
- And a direction $\mathbf{d} = (d_x, d_y, d_z)$
- Ray: $\mathbf{r} = \mathbf{o} + t\mathbf{d}$

$t \in \mathbb{R}^+$



Rays

- Example
  - Useful to store
  - Inverse direction

$$\mathbf{d}_{inv} = \frac{1}{\mathbf{d}}$$

- In Geometry.h

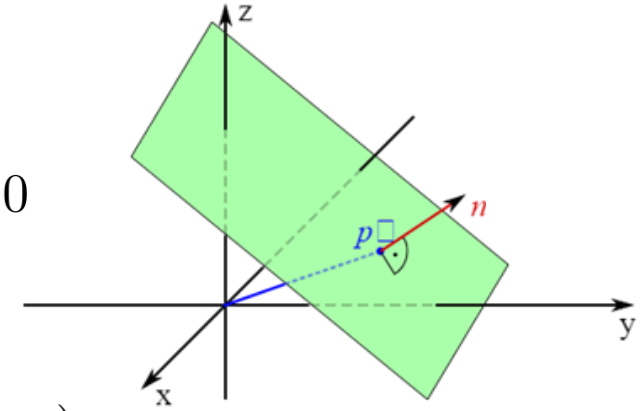
```
class Ray
{
public:
    Vec3 o;
    Vec3 dir;
    Vec3 invdir;
    Ray() {}
    Ray(const Vec3 _o, const Vec3 _dir) {
        init(_o, _dir);
    }
    void init(const Vec3 _o, const Vec3 _dir) {
        o = _o;
        dir = _dir;
        invdir = Vec3(1.0f, 1.0f, 1.0f) / dir;
    }
    Vec3 at(const float t) {
        return (o + (dir * t));
    }
};
```

Primitives

- Plane
- Triangle
- Box
- Sphere
- Others

Plane

- Parametric form $\mathbf{n} \cdot (\mathbf{P} - \mathbf{P}_0) = 0$
  - Normal $\mathbf{n} = (a, b, c)$
  - Point defining plane $\mathbf{P}_0 = (x_0, y_0, z_0)$
  - Arbitrary point on plane $\mathbf{P} = (x, y, z)$



$$a(x - x_0) + b(y - y_0) + c(z - z_0) = 0$$

$$ax + by + cz + d = 0$$

$$d = -(ax_0 + by_0 + cz_0)$$

Plane

- Ray-Plane Intersection
  - Insert ray equation into plane equation

$$a(o_x + td_x) + b(o_y + td_y) + c(o_z + td_z) + d = 0$$

  - Solve for t

$$t(ad_x + bd_y + cd_z) = -(ao_r + bo_y + co_z + d)$$

$$t = \frac{-(ao_x + bo_y + co_z + d)}{ad_x + bd_y + cd_z}$$

$$t = \frac{-(\mathbf{n} \cdot \mathbf{o} + d)}{\mathbf{n} \cdot \mathbf{d}}$$

Plane: Implementation

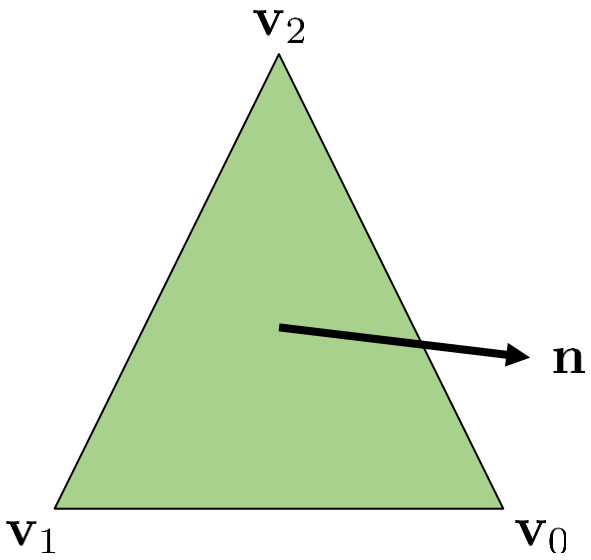
- Implement ray-plane intersection in Geometry.h

```
bool rayIntersect(Ray& r, float& t)
```

- Add test code

Triangles

- Three points $\mathbf{v}_i \in \mathbb{R}^3$
  - Normal $\mathbf{n}$
  - Attributes
    - uv coordinates
    - Shading normal
    - Tangent etc



Triangles

- Vertex Definition (as before)

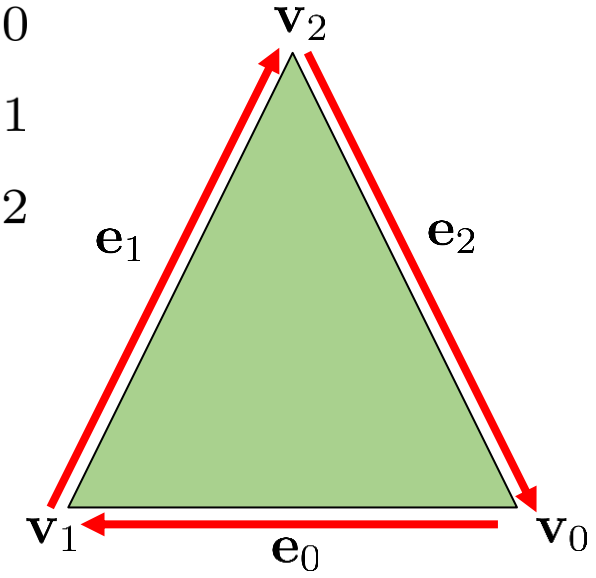
```
struct Vertex
{
        Vec3 p;
        Vec3 normal;
        float u;
        float v;
};
```

Triangles

- Edges: $\mathbf{e}_0 = \mathbf{v}_0 \rightarrow \mathbf{v}_1 = \mathbf{v}_1 - \mathbf{v}_0$

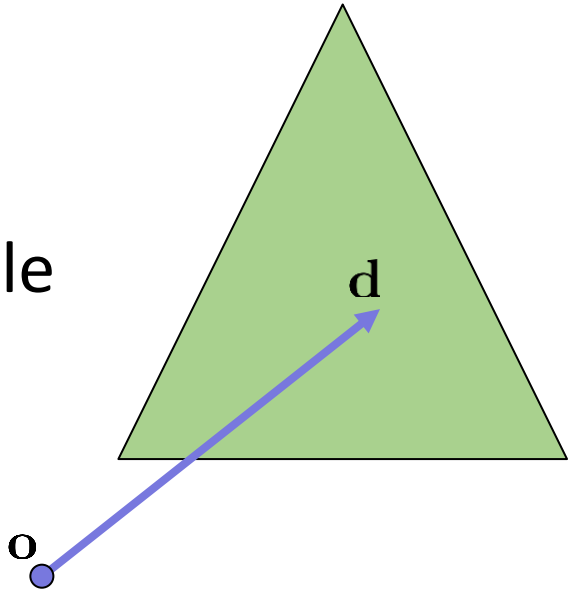
  $\mathbf{e}_1 = \mathbf{v}_1 \rightarrow \mathbf{v}_2 = \mathbf{v}_2 - \mathbf{v}_1$

  $\mathbf{e}_2 = \mathbf{v}_2 \rightarrow \mathbf{v}_0 = \mathbf{v}_0 - \mathbf{v}_2$



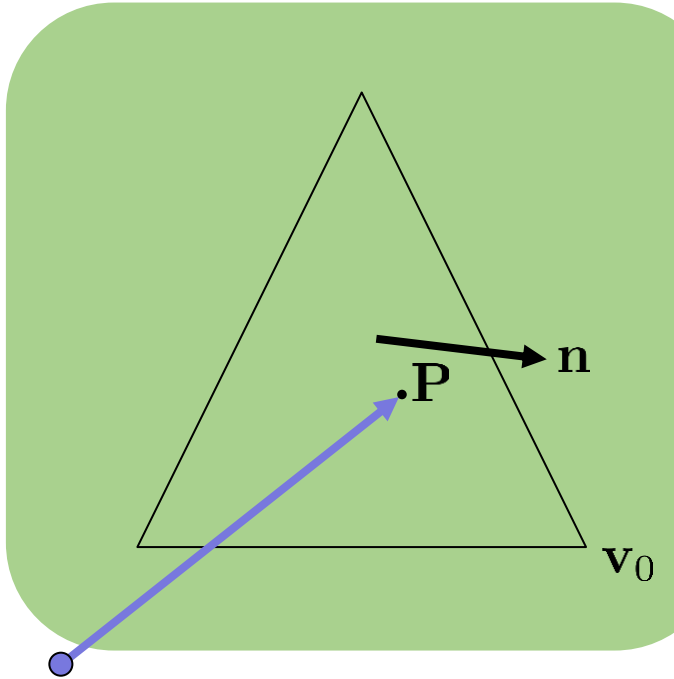
Triangles

- Ray Triangle
  - Many methods
  - Can build on rasterization
  - More efficient methods available



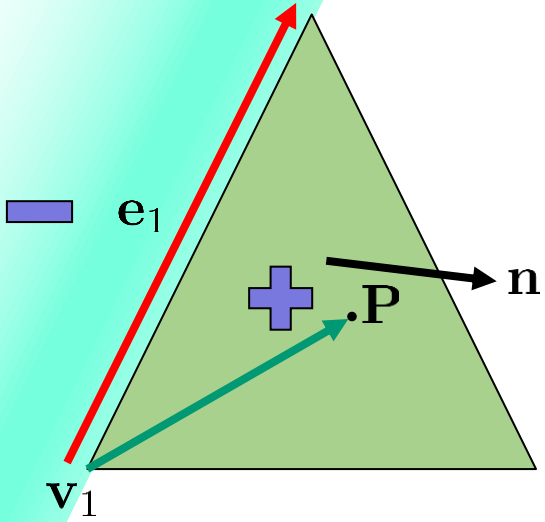
Triangles

- Ray Triangle (1)
  - Start with ray-plane
  - Compute $\mathbf{n} = (a, b, c)$
  - Find intersection point $\mathbf{P}$



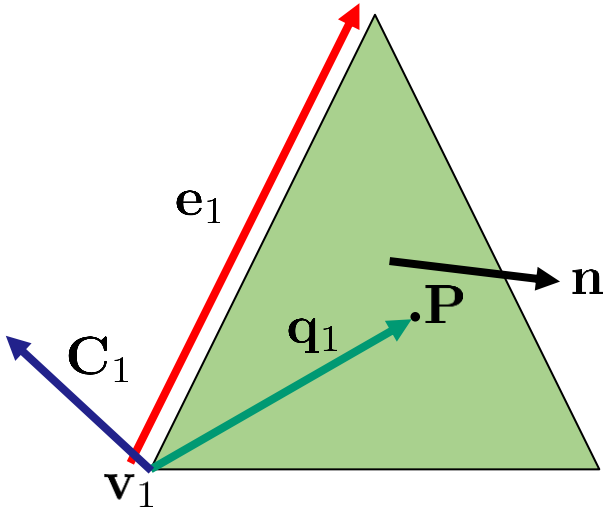
Triangles

- Ray Triangle
  - Check if $\mathbf{P}$ in triangle
  - Edge function as before
  - Cross product of edge
    - In 3D



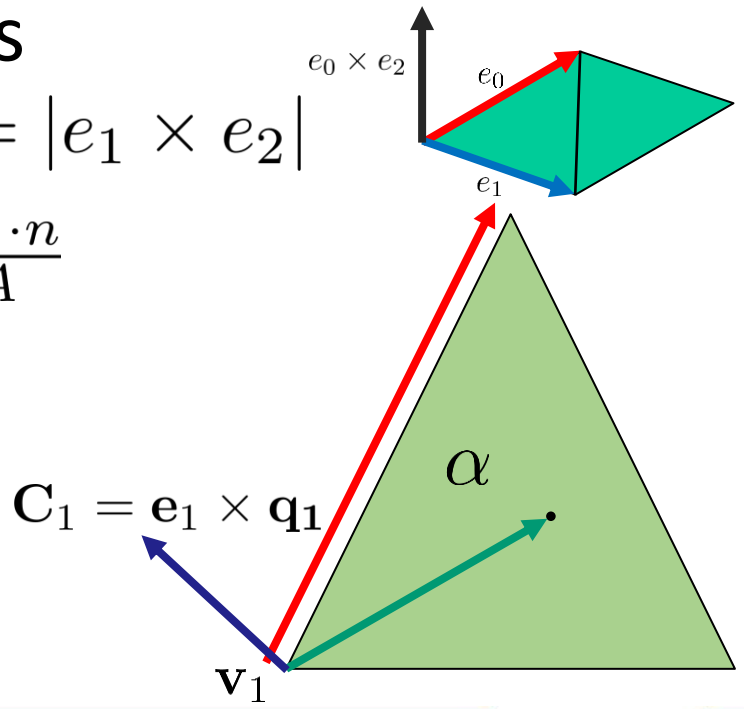
Triangles

- How to compute
  - Define vector to P: $\mathbf{q}_1 = \mathbf{P} - \mathbf{v}_1$
  - Cross product: $\mathbf{C}_1 = \mathbf{e}_1 \times \mathbf{q}$



Triangles

- Barycentric coordinates
  - Use area of triangle $A = |e_1 \times e_2|$
  - Normalize by A: $\alpha = \frac{\mathbf{C}_1 \cdot n}{A}$

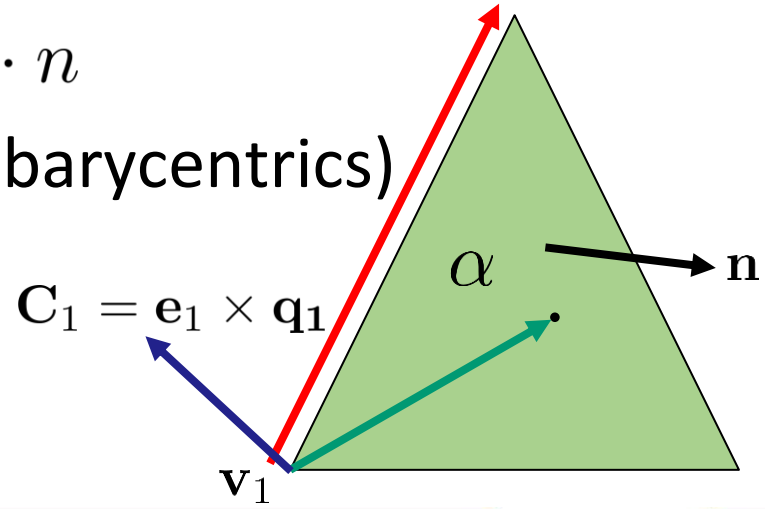


Triangles

- Barycentric coordinates
  - Faster area of triangle

$$A = (e_1 \times e_2) \cdot n$$

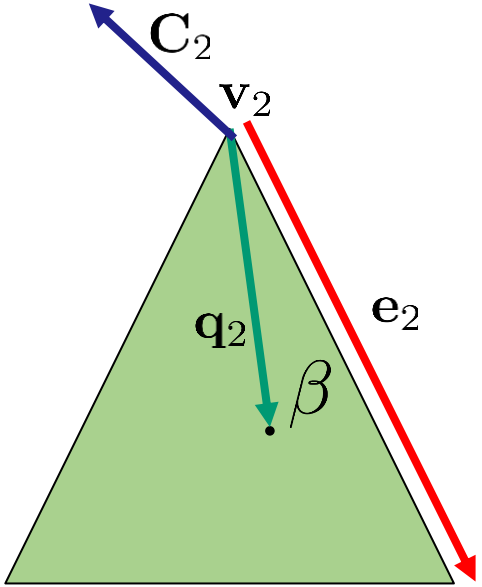
  - Includes sign (useful for barycentrics)
  - Avoids sqrt



Triangles

- Barycentric coordinates
  - Repeat for next edge $\mathbf{q}_2 = \mathbf{P} - \mathbf{v}_2$

    $\mathbf{C}_2 = \mathbf{e}_2 \times \mathbf{q}_2$
  - Compute $\beta = \frac{\mathbf{C}_2 \cdot n}{A}$



Triangles

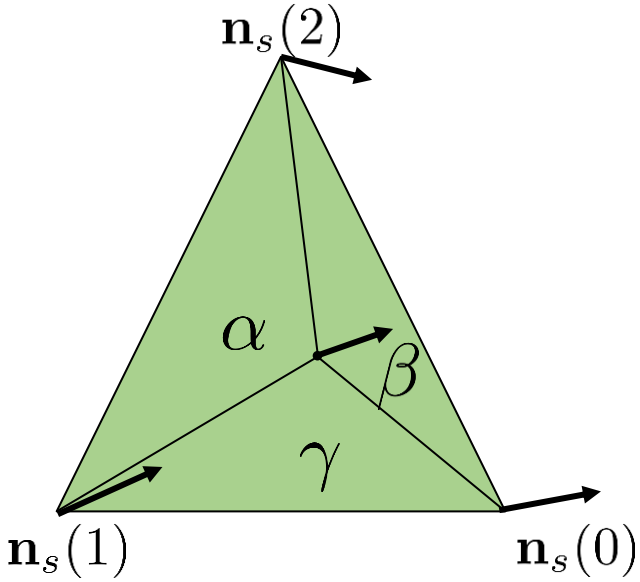
- Barycentric coordinates
  - Point in triangle if $\alpha \in [0..1]$
    $\beta \in [0..1]$
    $\alpha + \beta \leq 1$
  - Compute $\gamma = 1 - (\alpha + \beta)$

Triangles

- Barycentric coordinates
  - Can interpolate attributes
    - For shading normal $\mathbf{n}_s$

$$\mathbf{n}_s = \mathbf{n}_s(0)\alpha + \mathbf{n}_s(1)\beta + \mathbf{n}_s(2)\gamma$$

    - Needs to be normalized



Triangles

- Note helper functions

```
Vec3 centre() const

void interpolateAttributes(const float alpha, const float beta, const float gamma,
Vec3 &interpolatedNormal, float &interpolatedU, float &interpolatedV)
```

Triangles

- Add ray plane intersection in Triangle class in Geometry.h

```
bool rayIntersect(const Ray& r, float& t, float& u, float& v) const
```

  - May need to modify to add precomputed values

```
void init(Vertex v0, Vertex v1, Vertex v2, unsigned int _materialIndex)
```

- Add test code in Main.cpp

Triangles

- Making it more efficient
  - Solve ray-plane and point in triangle simultaneously
  - Moller-Trumbore
    - Directly solve for barycentric coordinates β, γ and distance t

Triangles

- Moller-Trumbore
  - Write intersection as equality $\mathbf{o} + t\mathbf{d} = \mathbf{v}_0 + \beta\mathbf{e}_1 + \gamma\mathbf{e}_2$
  - Rearrange $t\mathbf{d} = \mathbf{v}_0 - \mathbf{o} + \beta\mathbf{e}_1 + \gamma\mathbf{e}_2$
  - Define $\mathbf{T} = \mathbf{o} - \mathbf{v}_0$
  - Rewrite $-t\mathbf{d} + \beta\mathbf{e}_1 + \gamma\mathbf{e}_2 = \mathbf{T}$

Triangles

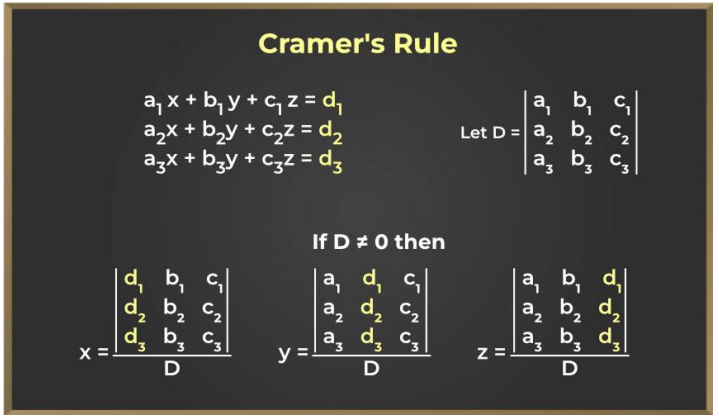
- Moller-Trumbore
  - Can write as system of equations

$$\begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & -\mathbf{d} \end{bmatrix} \begin{bmatrix} \beta \\ \gamma \\ t \end{bmatrix} = \mathbf{T}$$

  - Need

$$\begin{bmatrix} \beta \\ \gamma \\ t \end{bmatrix} = \begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & -\mathbf{d} \end{bmatrix}^{-1} \mathbf{T}$$

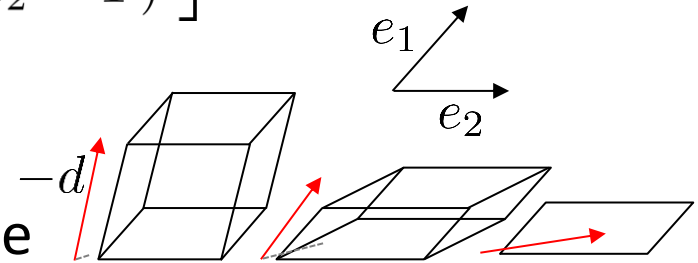
Triangles



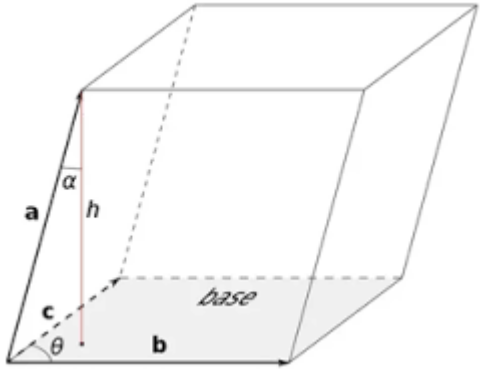
- Moller-Trumbore
  - Solve via Cramer's Rule

$$\begin{bmatrix} \beta \\ \gamma \\ t \end{bmatrix} = \frac{1}{det(\begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & -\mathbf{d} \end{bmatrix})} \begin{bmatrix} det(T & \mathbf{e}_2 & -\mathbf{d}) \\ det(\mathbf{e}_1 & T & -\mathbf{d}) \\ det(\mathbf{e}_1 & \mathbf{e}_2 & T) \end{bmatrix}$$

  - Need determinant:
    - Check if ray is parallel to triangle



Triangles



- Moller-Trumbore
  - Compute Determinant
    - Recall determinant encodes stretching of space due to transform

$$\det \begin{bmatrix} \mathbf{a} & \mathbf{b} & \mathbf{c} \end{bmatrix} = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$$

    - Cross: Creates vector perpendicular to b and c, area equal to formed parallelogram
    - Dot: Projects onto this computing the volume

Triangles

- Moller-Trumbore
  - Compute Determinant of $\begin{bmatrix} \mathbf{e}_1 & \mathbf{e}_2 & -\mathbf{d} \end{bmatrix}$:

$$\mathbf{p} = \mathbf{d} \times \mathbf{e}_2$$

$$\det = \mathbf{e}_1 \cdot \mathbf{p}$$

  - If $|\det| < \epsilon$ then ray parallel to plane
    - ϵ is a small value
    - e.g. 1e-6

Triangles

- Moller-Trumbore
  - Use determinant to solve for values (Cramer's Rule)

$$\beta = \frac{\mathbf{T} \cdot \mathbf{p}}{\det}$$

  - No intersection if $\beta < 0$ or $\beta > 1$

Triangles

- Moller-Trumbore
  - Use determinant to solve for values (Cramer's Rule)

$$\mathbf{q} = \mathbf{T} \times \mathbf{e}_1$$

$$\gamma = \frac{\mathbf{d} \cdot \mathbf{q}}{\det}$$

  - No intersection if $\gamma < 0$ or $\gamma > 1$ or $\beta + \gamma > 1$

Triangles

- Moller-Trumbore
  - Use determinant to solve for values (Cramer's Rule)

$$t = \frac{\mathbf{e}_2 \cdot \mathbf{q}}{\det}$$

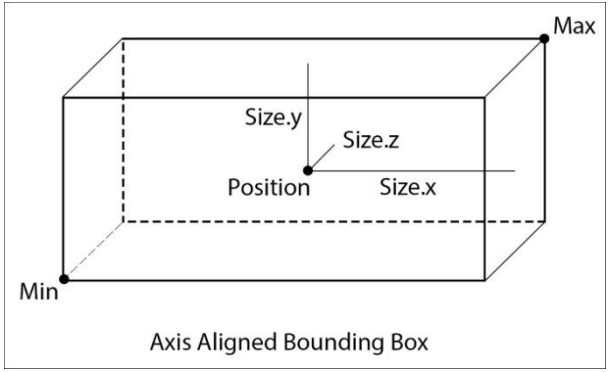
  - No intersection if $t < 0$

Triangles

- Implement Moller-Trumbore in your Triangle class
- Common optimization:
  - Store $\text{invdet} = \frac{1}{\det}$
  - Then reuse, e.g. $\beta = (\mathbf{T} \cdot \mathbf{p}) \cdot \text{invdet}$ instead of $\beta = \frac{\mathbf{T} \cdot \mathbf{p}}{\det}$
    - Multiplication cheaper than division

Axis Aligned Bounding Box

- AABB
- Used later to speed up computations
- Need fast Ray-AABB test
  - Defined as $\mathbf{B}_{\max} = (x_{\max}, y_{\max}, z_{\max})$
  - and $\mathbf{B}_{\min} = (x_{\min}, y_{\min}, z_{\min})$



Axis Aligned Bounding Box

- Ray-AABB
  - Check each plane of box (ray-plane)

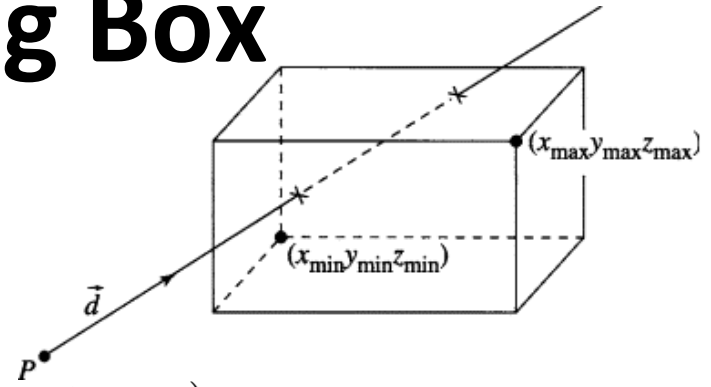
$$t_{\min,i} = \frac{B_{i,\min} - o_i}{d_i}$$

$$t_{\max,i} = \frac{B_{i,\max} - o_i}{d_i}$$

$$i \in \{x, y, z\}$$

Axis Aligned Bounding Box

- Ray-AABB
  - Find min and max t values



$$t_{\text{entry}} = \max(t_{\text{min},x}, t_{\text{min},y}, t_{\text{min},z})$$

$$t_{\text{exit}} = \min(t_{\text{max},x}, t_{\text{max},y}, t_{\text{max},z})$$

  - Intersects if $t_{\text{entry}} \leq t_{\text{exit}}$ and $t_{\text{exit}} \geq 0$
  - If $\mathbf{o}$ is inside the box $t_{\text{entry}} < 0$
    - Need to handle this sometimes

Axis Aligned Bounding Box

- Ray-AABB
  - Can check all six planes simultaneously
    - Can do this because box is axis aligned

$$\mathbf{t}_{\min} = (\mathbf{b}_{\min} - \mathbf{o})\mathbf{d}_{inv}$$

$$\mathbf{t}_{\max} = (\mathbf{b}_{\max} - \mathbf{o})\mathbf{d}_{inv}$$

    - Simultaneous test

$$\mathbf{t}_{\text{entry}} = \min(\mathbf{t}_{\min}, \mathbf{t}_{\max})$$

$$\mathbf{t}_{\text{exit}} = \max(\mathbf{t}_{\min}, \mathbf{t}_{\max})$$

Axis Aligned Bounding Box

- Ray-AABB
  - Can check all six planes simultaneously
    - Ensure values represent entry and exit

$$\mathbf{t}_{\text{entry}} = \min(\mathbf{t}_{\min}, \mathbf{t}_{\max})$$

$$\mathbf{t}_{\text{exit}} = \max(\mathbf{t}_{\min}, \mathbf{t}_{\max})$$

    - Compute values

$$t_{\text{entry}} = \max(\mathbf{t}_{\text{entry}})$$

$$t_{\text{exit}} = \min(\mathbf{t}_{\text{exit}})$$

Axis Aligned Bounding Box

- Implement ray-AABB intersection in AABB class
  - Two versions:

```
bool rayAABB(const Ray& r)
bool rayAABB(const Ray& r, float& t)
```

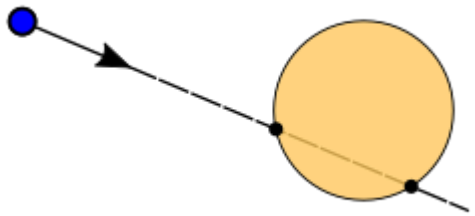
- Also need to compute area of AABB for later

```
float area()
```

- Add tests to Main.cpp

Sphere

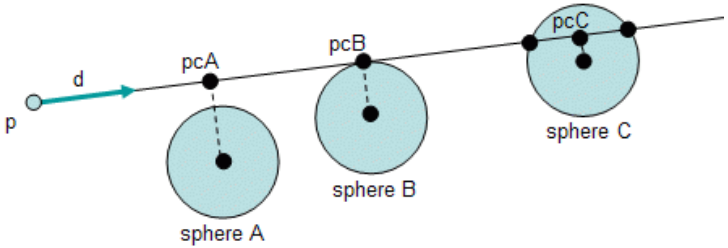
- Defined by $\|\mathbf{p} - \mathbf{c}\|^2 = R^2$
  - Centre $\mathbf{c}$
  - Radius R



- Substitute ray equation $\|(\mathbf{o} + t\mathbf{d}) - \mathbf{c}\|^2 = R^2$
  - Define $\mathbf{l} = \mathbf{o} - \mathbf{c}$ so $\|\mathbf{l} + t\mathbf{d}\|^2 = R^2$
  - Expand $\mathbf{l} \cdot \mathbf{l} + 2t(\mathbf{l} \cdot \mathbf{d}) + t^2(\mathbf{d} \cdot \mathbf{d}) = R^2$

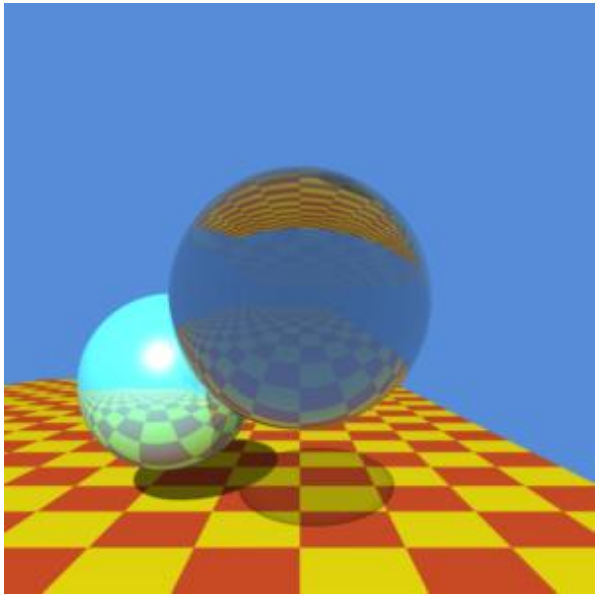
Sphere

- Ray-Sphere
  - Simplify $t^2 + 2(\mathbf{l} \cdot \mathbf{d})t + (\mathbf{l} \cdot \mathbf{l} - R^2) = 0$ as $\mathbf{d} \cdot \mathbf{d} = 1$
  - Quadratic Equation $a = 1, \quad b = \mathbf{l} \cdot \mathbf{d}, \quad c = \mathbf{l} \cdot \mathbf{l} - R^2$
  - Calculate discriminant $dis = b^2 - c$
    - If $dis > 0$ two intersections
    - If $dis = 0$ one intersection
    - If $dis < 0$ no intersection



Sphere

- Ray-Sphere
  - Solve for t: $t = -b \pm \sqrt{dis}$
  - Need to handle $\mathbf{o}$ inside sphere



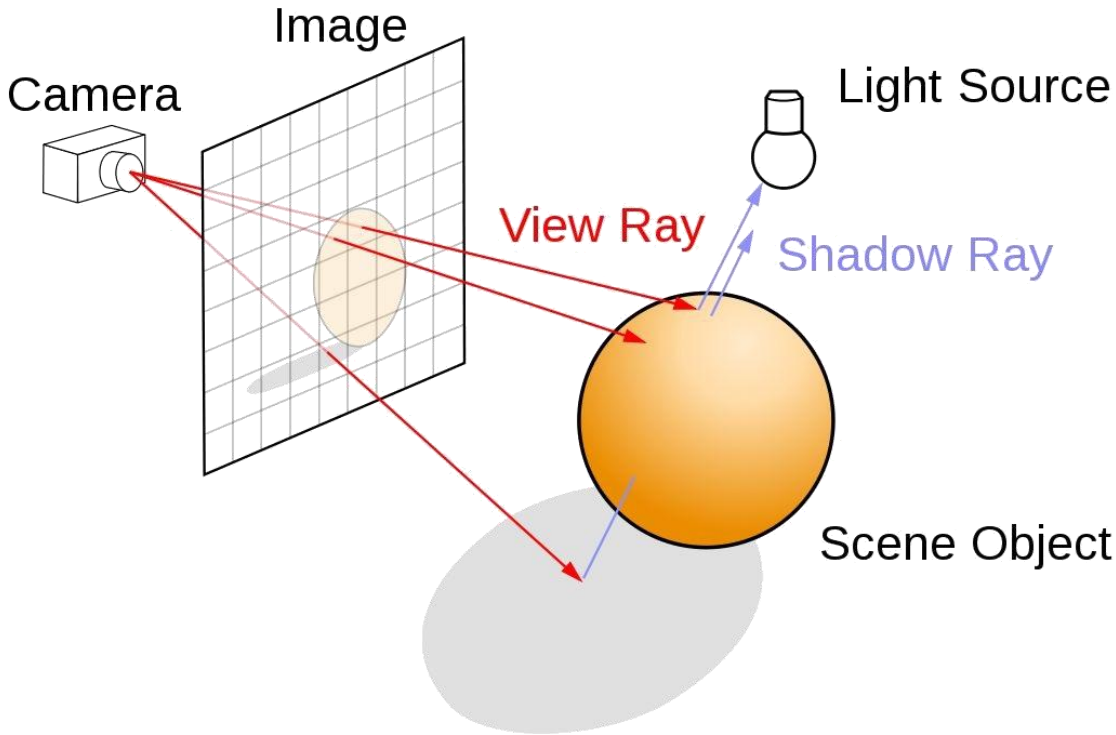
Sphere

- Implement ray-sphere intersection in the Sphere class in Geometry.h

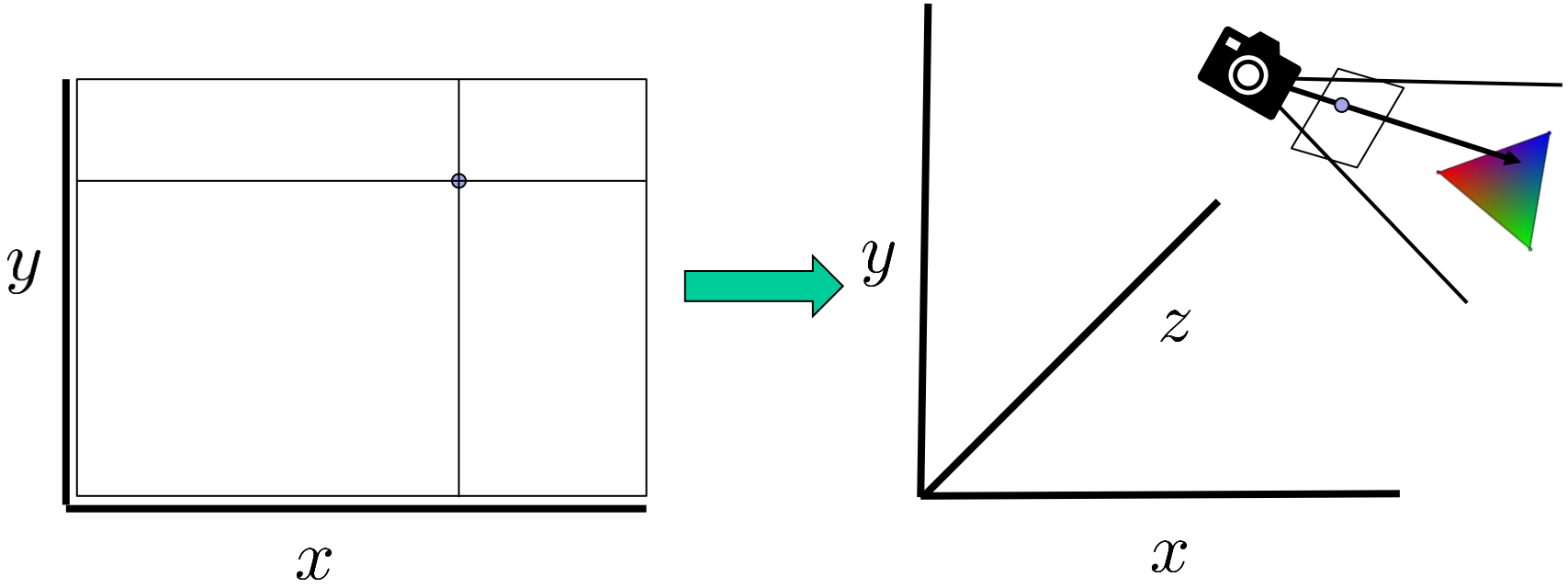
```
bool rayIntersect(Ray& r, float& t)
```

- Add tests in Main.cpp

Generating Rays

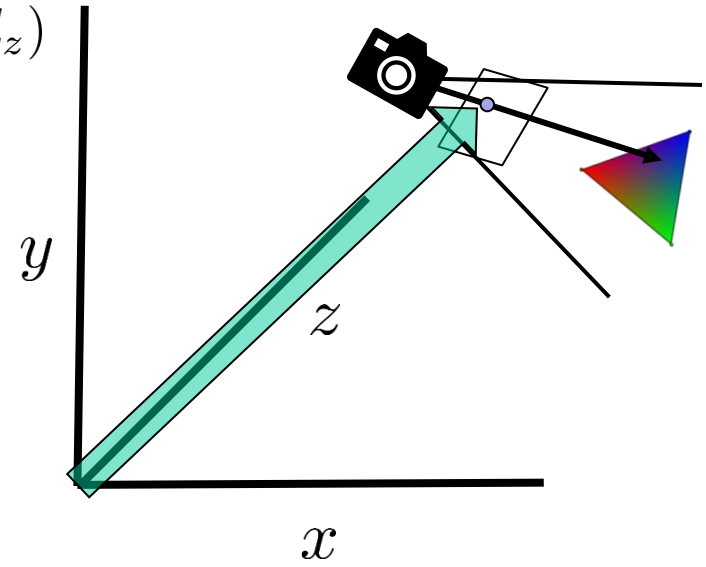


Generating Rays



Generating Rays

- Need ray origin $\mathbf{o} = (o_x, o_y, o_z)$
- And ray direction $\mathbf{d} = (d_x, d_y, d_z)$
  - In world space



Generating Rays

- What we have
  - View matrix (world -> camera): $\mathbf{V}$
  - Projection matrix (camera -> clip space): $\mathbf{P}$
- What we want:
  - Clip space -> camera
  - Camera -> world

Generating Rays

- What we have
  - View matrix (world -> camera): $\mathbf{V}$
  - Projection matrix (camera -> clip space): $\mathbf{P}$
- What we want:
  - Clip space -> camera (inverse $\mathbf{P}$): $\mathbf{P}^{-1}$
  - Camera -> world (inverse $\mathbf{V}$): $\mathbf{V}^{-1}$

Generating Rays

- Ray origin $\mathbf{o} = (o_x, o_y, o_z)$
- Either
  - Set as camera position
  - Extract translation from $\mathbf{V}^{-1}$
  - Compute $\mathbf{o} = \mathbf{V}^{-1}\mathbf{O}$
    - Where $\mathbf{O} = (0, 0, 0)$

Generating Rays

- Ray direction
  - Needs to project from point on film into world
  - Split into two:
    - Unproject from camera
    - Transform into world

Generating Rays

- Given a point on the film

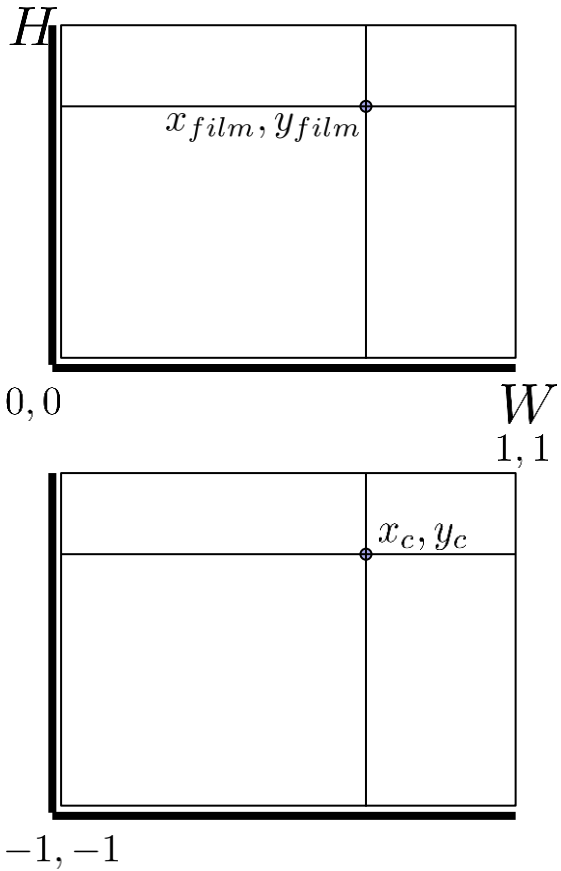
$$x_{film} \in [0..W]$$

$$y_{film} \in [0..H]$$

- Normalize to NDC

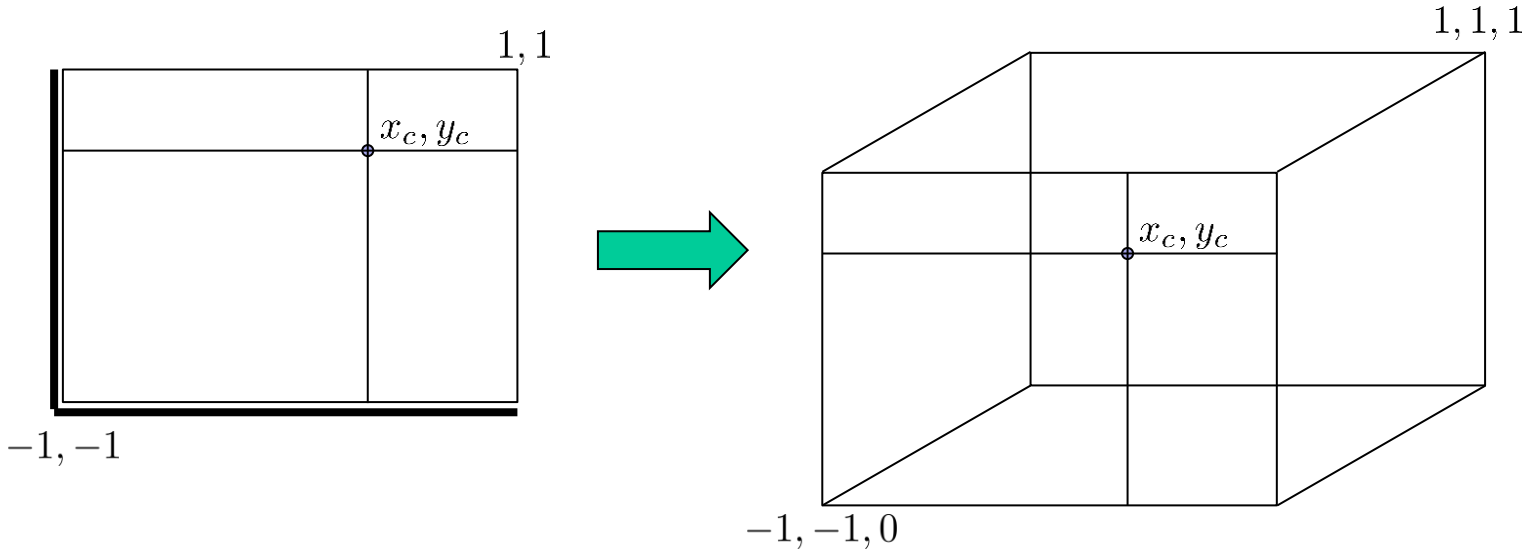
$$x_c = \frac{2x_{film}}{W} - 1$$

$$y_c = \frac{2y_{film}}{H} - 1$$



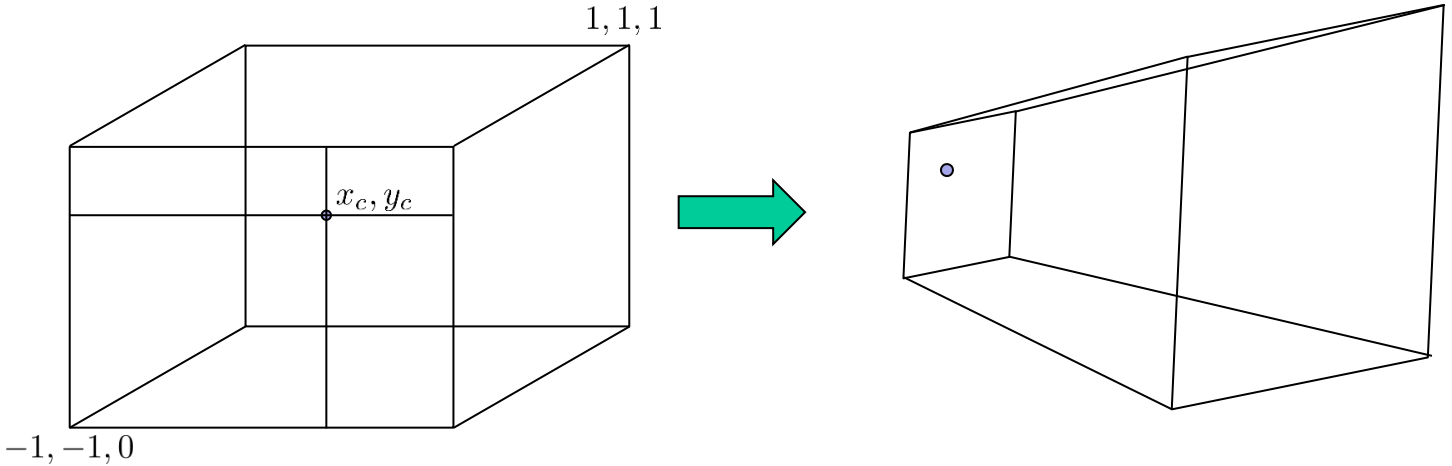
Generating Rays

- NDC to Clip Space $\mathbf{p}_{clip} = (x_c, y_c, 0, 1)$



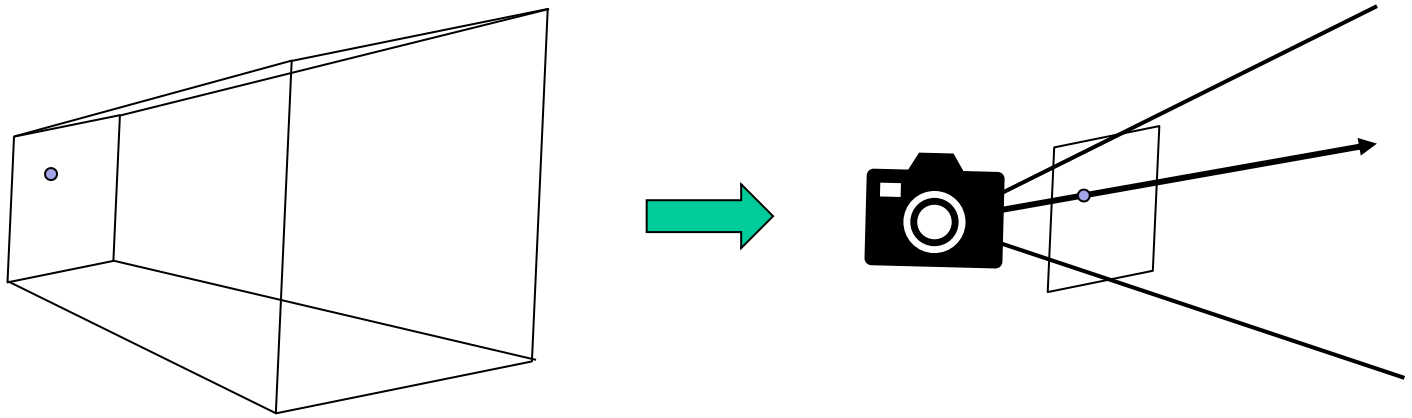
Generating Rays

- Clip Space to camera space $\mathbf{p}_{camera} = \mathbf{P}^{-1}\mathbf{p}_{clip}$
  - Gives homogenous coordinate



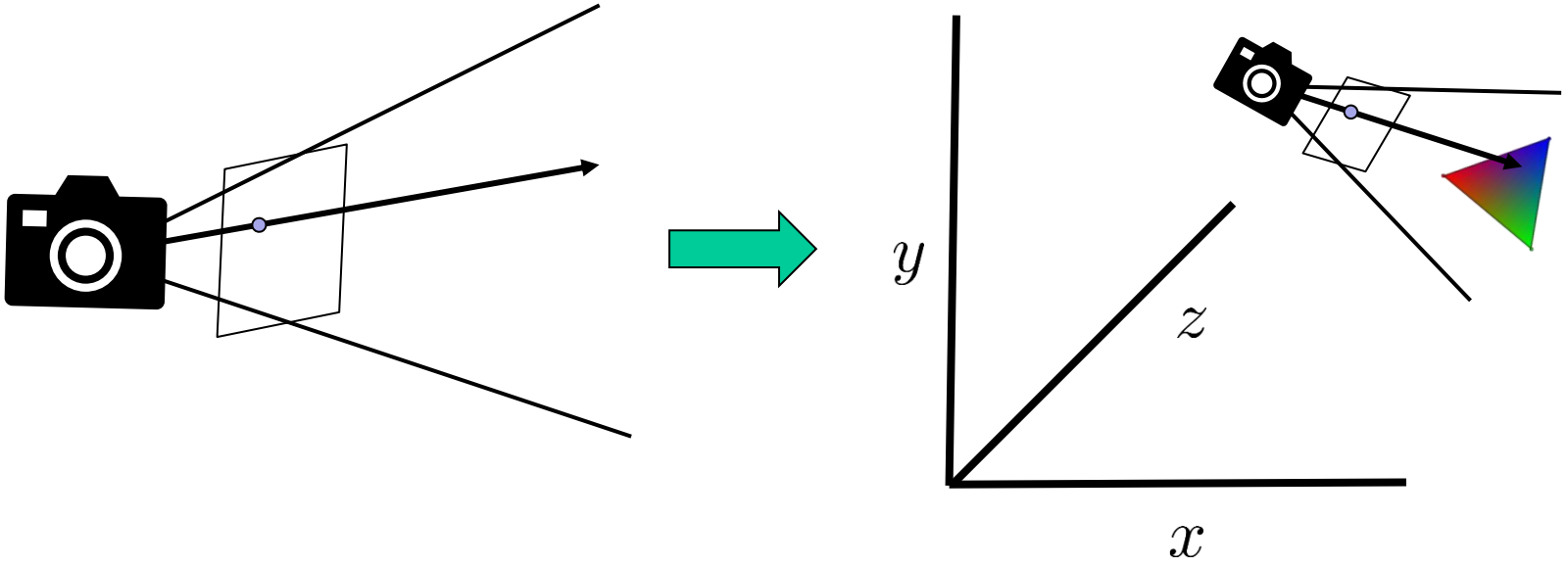
Generating Rays

- Homogenous to camera space
  - Need perspective division $\mathbf{d}_{camera} = \frac{\mathbf{p}_{camera}(x, y, z)}{\mathbf{p}_{camera}(w)}$



Generating Rays

- Camera space to world space $\mathbf{d} = \mathbf{V}^{-1}\mathbf{d}_{camera}$



Generating Rays

- Implement camera ray generation in Camera class in Scene.h

```
Ray generateRay(float x, float y)
```

– Will need:

```
Matrix inverseProjectionMatrix;
Matrix camera;
float width;
float height;
```

$\mathbf{P}^{-1}$

$\mathbf{V}^{-1}$

– Origin:

```
Vec3 origin;
```

Generating Rays

- Implement camera ray generation in Camera class in Scene.h
  - Matrix method performs multiplication and perspective division

```
Vec3 mulPointAndPerspectiveDivide(const Vec3& v)
```

Image Generation

- Iterate over all pixels in Raytracer class in Renderer.h
  - Two for loops

```
void render()
{
    for (int y = 0; y < camera->height; y++) {
        for (int x = 0; x < camera->width; x++) {
            // Ray tracing happens here!
        }
    }
}
```

  - Can be parallelized (will discuss later)

Image Generation

- Pick a point in the pixel (e.g. pixel centre)

```
float px = x + 0.5f;
float py = y + 0.5f;
Ray ray = scene->camera.generateRay(px, py);
```

- Will generalize later

Image Generation

- Iterate over all pixels ✓
- Pick a point in the pixel ✓
- Generate rays ✓
- Intersect scene geometry $\frac{1}{2}$
- Shade ✗

Intersect Scene Geometry

- So far we have primitives
- Need to find nearest intersection
  - $t_{actual} = min(t_i), i \in [primitives]$
- Iterate over all triangles
- Find intersection
- Store attributes of nearest intersection

Intersect Scene Geometry

- Attributes
  - Triangle ID id
  - Intersection data (t and barycentric) $(t_{id}, \alpha, \beta, \gamma)$

```
struct IntersectionData
{
        unsigned int ID;
        float t;
        float alpha;
        float beta;
        float gamma;
};
```

Intersect Scene Geometry

- In Scene class

```
IntersectionData traverse(const Ray& ray) {
    IntersectionData intersection;
    intersection.t = FLT_MAX;
    for (int i = 0; i < triangles.size(); i++) {
        float t; float u;  float v;
        if (triangles[i].rayIntersect(ray, t, u, v)) {
            if (t < intersection.t) {
                intersection.t = t;
                intersection.ID = i;
                intersection.alpha = u;
                intersection.beta = v;
                intersection.gamma = 1.0f - (u + v);
            }
        }
    }
    return intersection;
}
```

Intersection Data

- Will need data for shading and lighting

```
class ShadingData
{
public:
        Vec3 x;
        Vec3 wo;
        Vec3 sNormal;
        Vec3 gNormal;
        float tu;
        float tv;
        Frame frame;
        BSDF* bsdf;
        float t;
};
```

Intersection Data

- Construct from `IntersectionData`
  - In Scene class

```
ShadingData calculateShadingData(IntersectionData intersection, Ray& ray)
```

- Fills in details
  - Position of intersections, Shading and Geometric Normal
  - Material information
  - Texture coordinates etc

First Raytracer

- Visualize normals

```
Colour viewNormals(Ray& r)
{
    IntersectionData intersection = scene->traverse(r);
    if (intersection.t < FLT_MAX)
    {
        ShadingData shadingData = scene->calculateShadingData(intersection, r);
        return Colour(fabsf(shadingData.sNormal.x),
fabsf(shadingData.sNormal.y), fabsf(shadingData.sNormal.z));
    }
    return Colour(0.0f, 0.0f, 0.0f);
}
```

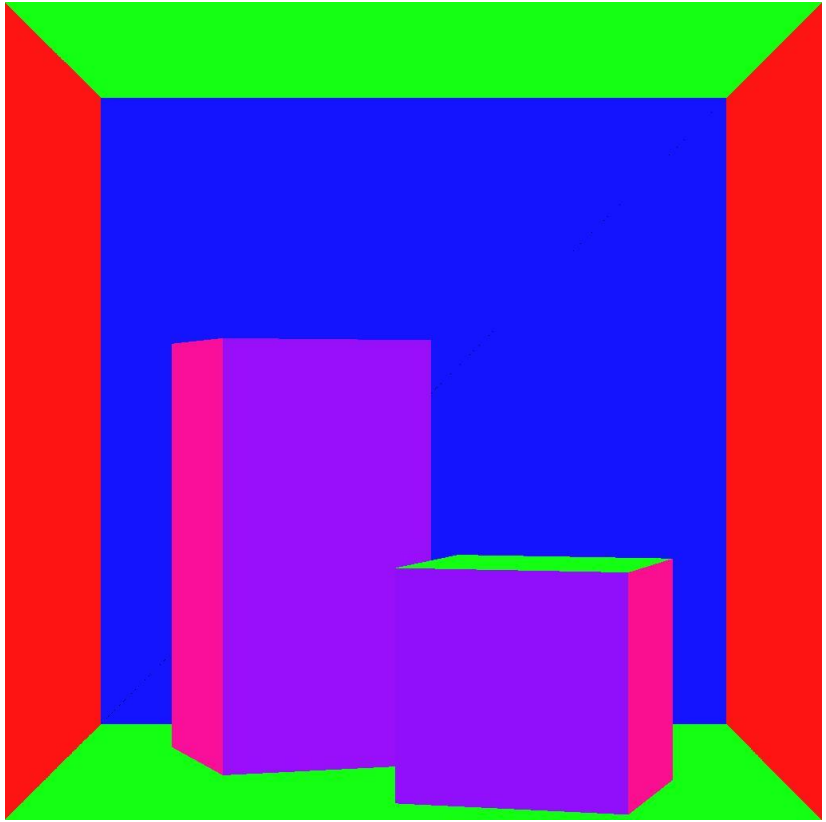
First Raytracer

- Call from

```
void render()
{
    film->incrementSPP();
    for (int y = 0; y < film->height; y++) {
        for (int x = 0; x < film->width; x++) {
            float px = x + 0.5f;
            float py = y + 0.5f;
            Ray ray = scene->camera.generateRay(px, py);
            Colour col = viewNormals(ray);
            unsigned char r = col.r * 255;
            unsigned char g = col.g * 255;
            unsigned char b = col.b * 255;
            canvas->draw(x, y, r, g, b);
        }
    }
}
```

First Raytracer

- Visualize normals



First Raytracer

- View surface colours

```
Colour albedo(Ray& r)
{
    IntersectionData intersection = scene->traverse(r);
    ShadingData shadingData = scene->calculateShadingData(intersection, r);
    if (shadingData.t < FLT_MAX)
    {
        if (shadingData.bsdf->isLight())
        {
            return shadingData.bsdf->emit(shadingData, shadingData.wo);
        }
        return shadingData.bsdf->evaluate(shadingData, Vec3(0, 1, 0));
    }
    return scene->background->evaluate(shadingData, r.dir);
}
```

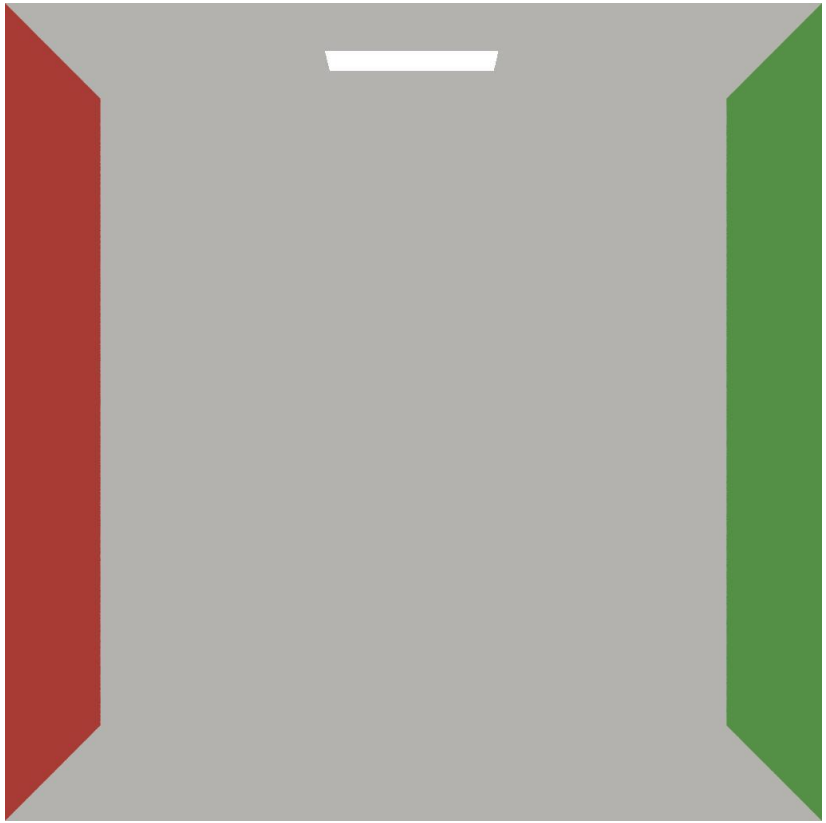
First Raytracer

- Call from

```
void render()
{
    film->incrementSPP();
    for (int y = 0; y < film->height; y++) {
        for (int x = 0; x < film->width; x++) {
            float px = x + 0.5f;
            float py = y + 0.5f;
            Ray ray = scene->camera.generateRay(px, py);
            Colour col = albedo(ray);
            unsigned char r = col.r * 255;
            unsigned char g = col.g * 255;
            unsigned char b = col.b * 255;
            canvas->draw(x, y, r, g, b);
        }
    }
}
```

First Raytracer

- View surface colours

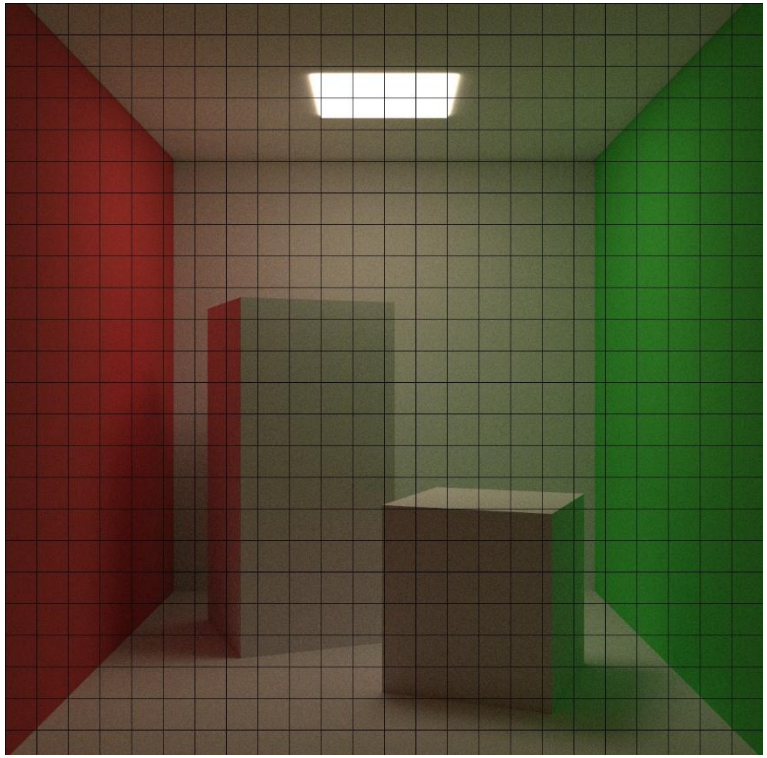


Parallel Computation

- Ray tracing is embarrassingly parallel
- Pixel / Sample computation independent across screen
- But computational costs an vary per ray
  - More geometry
  - More complicated shading etc

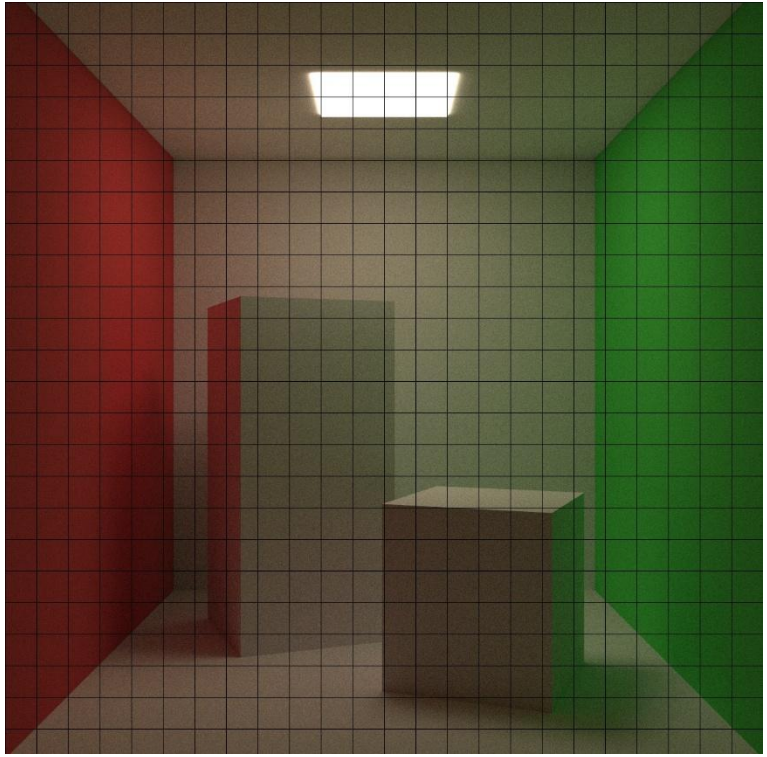
Parallel Computation

- Tile based rendering
  - Split screen into a grid of tiles
  - Process tile in parallel
  - What size?
  - Why not 1x1?



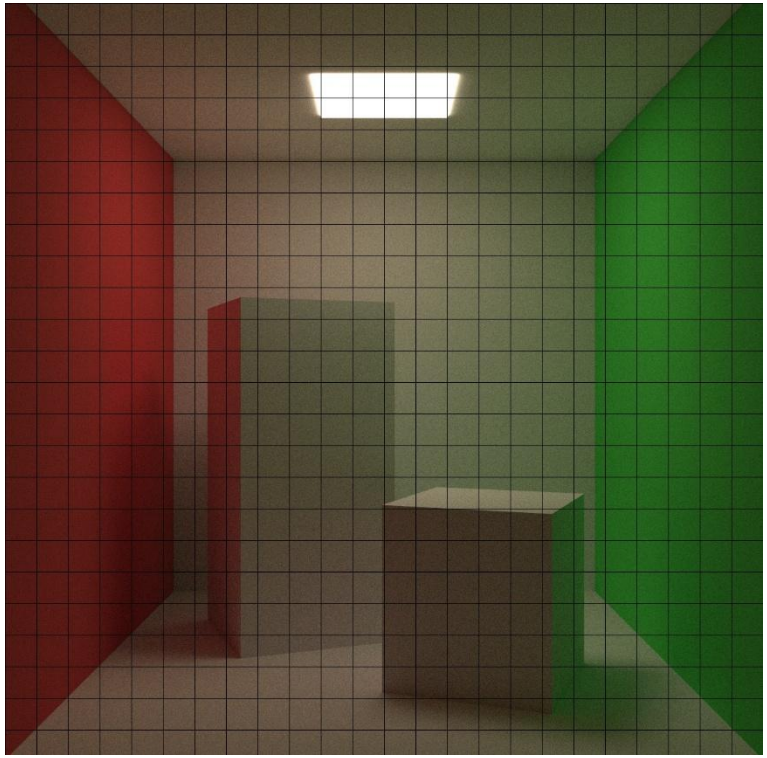
Parallel Computation

- Tile based rendering
  - Job queue of tile IDs
  - Processed using multiple threads
  - Atomic increment tile ID being processed per thread



Parallel Computation

- Assignment!
- Start now!



Questions?

Advanced Computer Graphics

Ray Tracing

Dr. Tom Bashford-Rogers